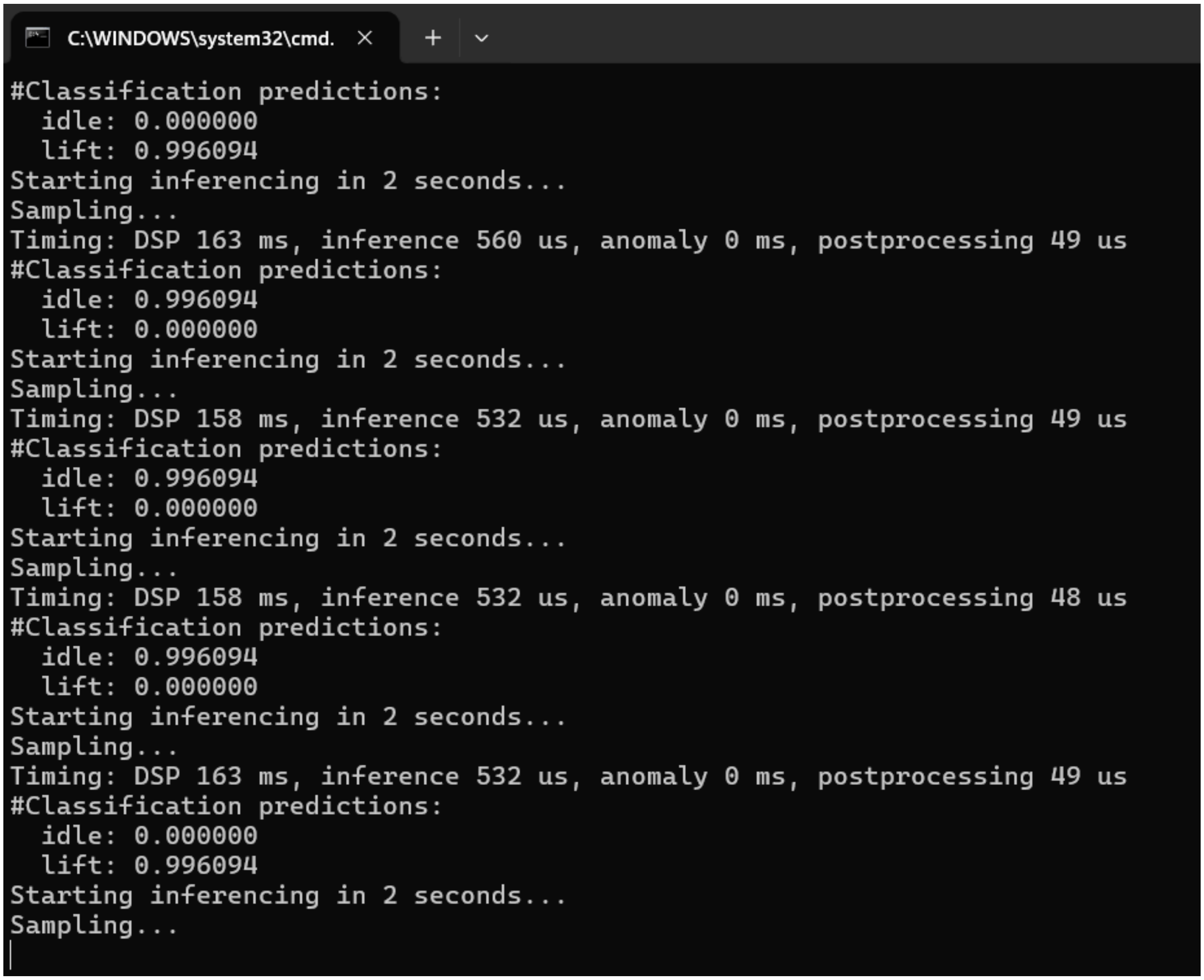
EE 446 Lab 7
Zelamir Reynolds-Henkins

Edge Impulse Link: https://studio.edgeimpulse.com/public/1080748/live

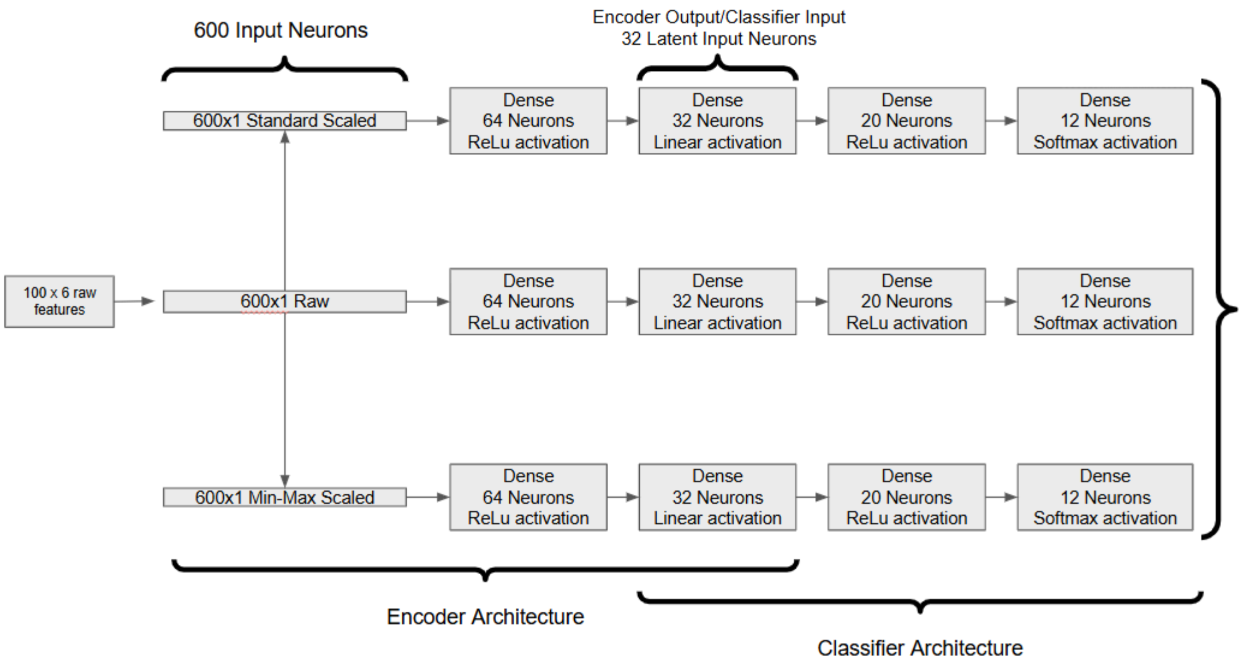
Part 1: Edge Impulse model predictions using CLI

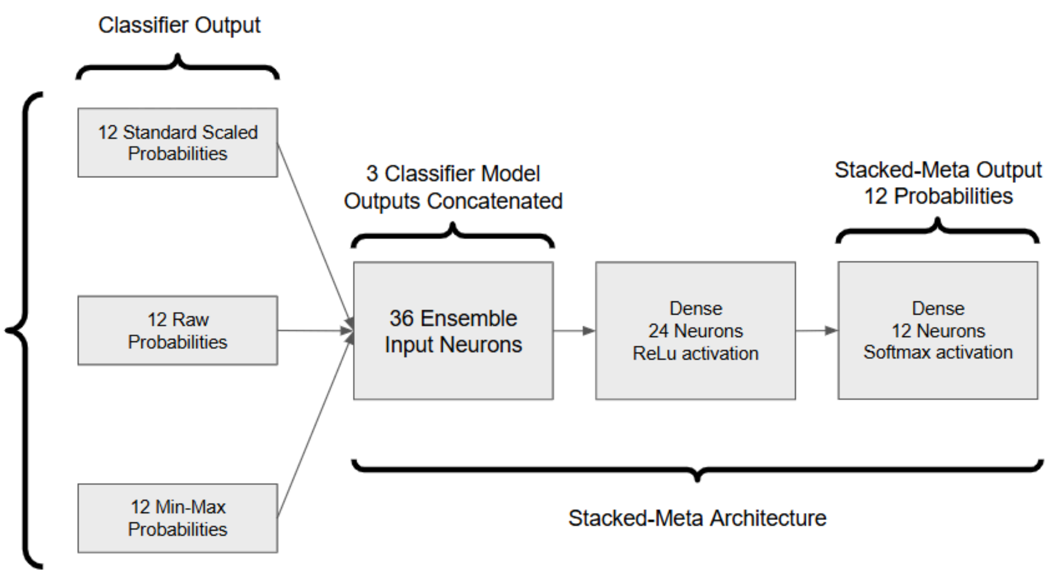


```
C:\WINDOWS\system32\cmd.
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 560 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 158 ms, inference 532 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 158 ms, inference 532 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 532 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
```

Part 2:

1. Model Architecture





2. a) Pruning is applied using polynomial decay over 500 steps, with initial sparsity of 0.1 and a final sparsity of 0.8.
   b) The wrappers are stripped before QAT so the quantization doesn't stack with the pruning wrappers
   c) QAT is applied by simulating quantization effects during training using TFMot fine-tuning over 5 epochs

d) If masks aren't reapplied, the optimizer may update zero-value weights, which would negate the sparsity applied. Reapplying masks preserves sparsity.
e) Representative data is needed to calibrate quantization ranges using actual samples.
f) Pruning reduces the amount of non-zero weights so only the most important weights are preserved, therefore reducing the size of the model. It does so by rounding down/eliminating the weights that are already near zero. Int8 quantization reduces the precision of the leftover weights, so each one is rounded, thus reducing the model size further.

3. The serial monitor shows that while the arduino is at rest, the model predicts "Laying down", which is the correct classification while no motion is present. It shows the probabilities for each classification from each model, and all have laying down (index 3) as the most probable. However the min-max and standard-scaled models were less confident. It is odd that those models have relatively high prediction values that the arduino is climbing stairs or jogging rather than sitting and relaxing or standing still. Placing the arduino on a pillow or something that absorbs vibrations may make this prediction more accurate.

4. The model did not predict actions involving motion very consistently. The orientation of the arduino and where/how on the body it is mounted seem to strongly influence the classification output. Holding it still in different orientations does seem to return different classifications (Laying down, sitting and relaxing, standing still). It really depends on exactly how the model was trained. If it was attached to an ankle vs around the waist vs in hand, and at what orientation is vitally important to how the model classifies actions.